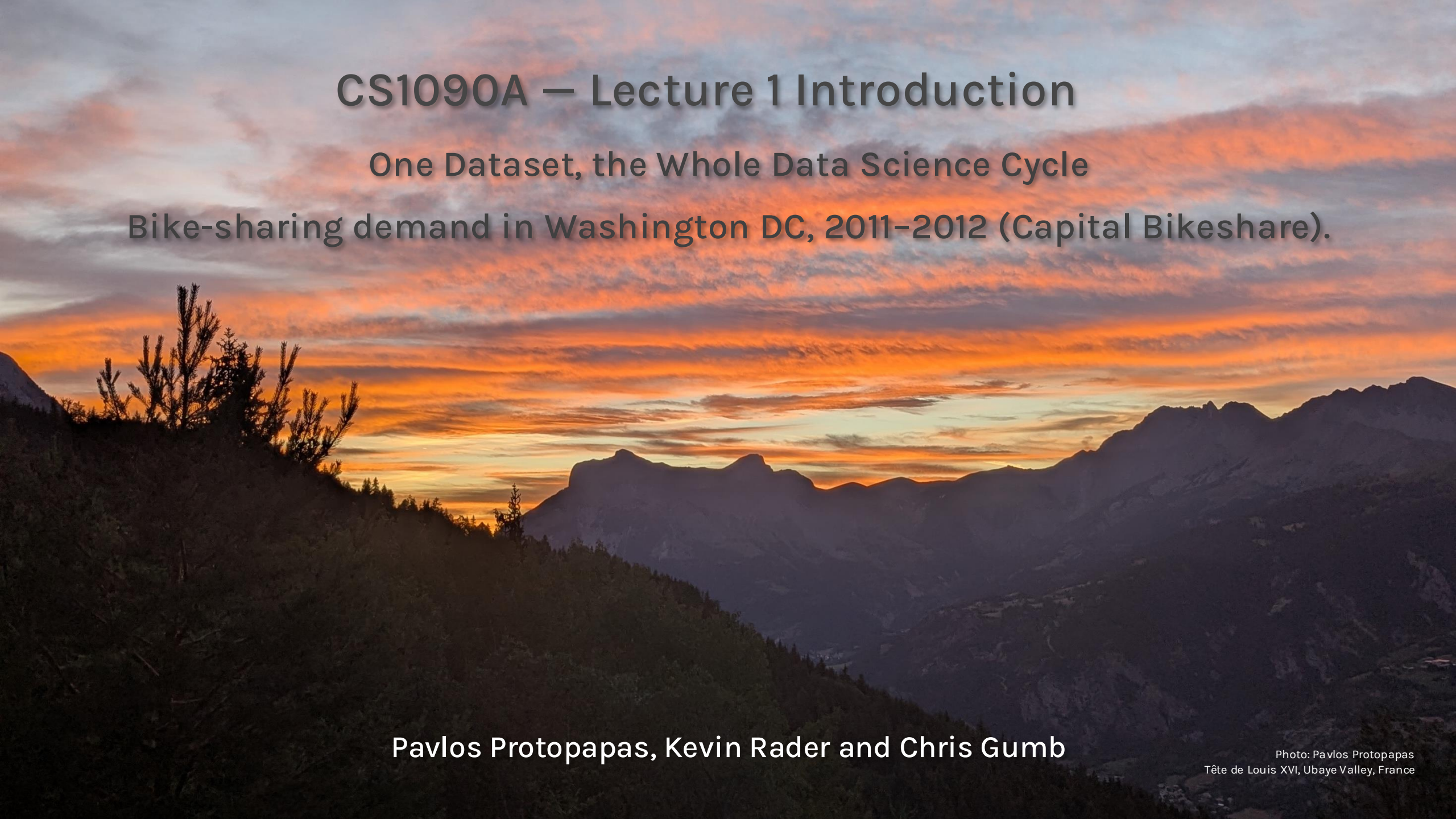
CS109OA — Lecture 1 Introduction

One Dataset, the Whole Data Science Cycle

Bike-sharing demand in Washington DC, 2011–2012 (Capital Bikeshare).



Pavlos Protopapas, Kevin Rader and Chris Gumb

Photo: Pavlos Protopapas
Tête de Louis XVI, Ubaye Valley, France

PART ONE

Who We Are

Pavlos Protopapas



Instructor

School of Engineering and Applied Sciences (SEAS)

Scientific Director for the Data Science and Computational Science & Engineering master's programs.

Stellar DNN: AI for Science—Physics-informed neural networks, astronomy and black holes, language models for scientific literature.



Funny fact: during a failed military service, he was declared the worst soldier in NATO. He's a certified cook (Le Cordon Bleu!) but loves eating more than cooking.

Kevin Rader



Instructor

Department of Statistics

Senior Preceptor and Associate Director of Undergraduate Studies in Statistics.

Teaches Stat courses from intro-level (Stat 104) through biostatistical methods (Stat 116), and advises undergraduate research applying data science to sports, medicine, and policy.

⚽ *Fun fact: coaches 5th-grade girls' soccer — 0-1 so far, and soccer analytics haven't helped. A die-hard Philadelphia sports fan (go Birds!) who'd rather grow his own food than be on TikTok.*



Chris Gumb



Preceptor

School of Engineering and Applied Sciences (SEAS)

Has been a member of the CS109A/B teaching staff for the past 9 years.

As preceptor, teaches lectures, coordinates the TF team, develops course materials, and handles logistics.



When he's not answering your Ed posts and emails, he's making music or seeing films with friends — frequently spotted at local indie theaters. He's basically made of popcorn.

Course Staff at a Glance

- Pavlos Protopapas — Instructor (SEAS)
- Kevin Rader — Instructor (Department of Statistics)
- Chris Gumb — Preceptor (SEAS)
- A team of Teaching Fellows (TFs) supports sections, office hours, and project mentoring — roster to be posted on Canvas / Ed.



PART TWO

Course Philosophy

Build the foundation first.

Why Start Here in the Era of AI?

- AI can write the model.
- You still need to know when it's wrong.
- This course builds that judgment—with real, messy data and hands-on practice.



The Path to Understanding AI



This course lives in the first three steps — data, wrangling, and storytelling — plus a first pass at prediction and statistical learning.

Every step to the right of "Machine Learning" is built on top of what you learn here. Skip the foundation, and the later steps become tricks you can't debug.

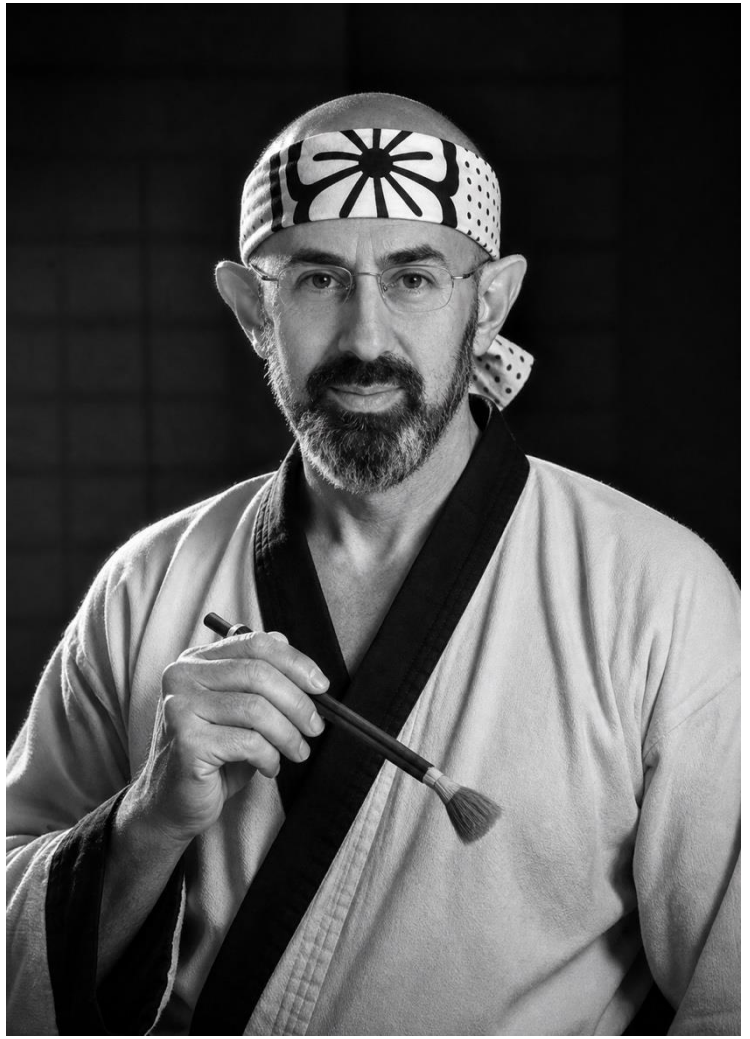
Wax On, Wax Off

In The Karate Kid, Daniel waxes cars and sands floors before he learns a single strike.

This course works the same way.

Cleaning messy data, fitting a simple regression, and computing an interval by hand may not feel like "real AI."

They are the habits that make real AI possible.



Foundations That Still Matter

- Random forests, gradient boosting, and other classical ensembles still win on tabular data — the kind you'll meet most often outside a research lab.
- Linear and logistic regression remain the most interpretable tools for explaining why a model made a decision, not just what it decided.
- Cross-validation, bootstrapping, and hypothesis testing are how you know whether a result is real — skills no amount of prompting replaces.
- These methods are durable. They will still be the right tool for large classes of problems long after any specific AI system has been replaced.

Foundations That Still Matter

- Random forests & gradient boosting still win on tabular data.
- Linear & logistic regression explain why a model decided.
- Cross-validation, bootstrapping & testing separate signal from noise.

Durable methods. Better judgment. No amount of prompting replaces either.

PART THREE

A Data Science Use Case

From raw data to a decision

Our Case Study: Bike Sharing, D.C.

Capital Bikeshare is Washington, D.C.'s public bike-sharing system: riders unlock a bike at one station and return it at another.

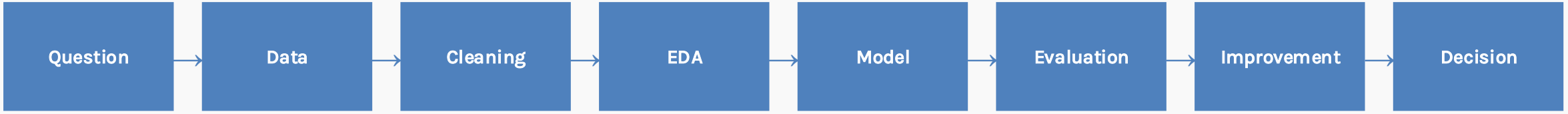
Our dataset records two years of that activity, 2011–2012, one row per hour.

It captures both real-world behavior (how many people rode) and the conditions around it — weather, season, day of week.

We will not learn every detail yet. First, we will see the whole journey once.

The roadmap: the data science cycle

Every project in this field moves through the same cycle. Today we walk it once, start to finish, on one dataset.



We will come back to this exact slide at the end of the lecture.

The questions we are asking

THE QUESTION WE ARE BEING PAID TO ANSWER

It is a spring evening in 2012. Overnight we decide how many bikes to put on the street tomorrow. **How many — and at which hours?**

Not "can we build a model." To answer it we have to predict demand — and, harder, know when the prediction is wrong.

Five guesses we will test along the way — commit to each one before we look:

hypothesis	expected effect
Warmer weather	more rentals
Very hot weather	maybe fewer — the relationship bends back down
Rush hour	more rentals
Weekends	a different shape of day
Small subgroups	can mislead you

Write these down. At the end of the lecture we come back to this list and mark it.

What the columns mean

column	meaning
`dteday`, `hr`	date and hour of day (0–23)
`temp`, `atemp`	temperature and "feels like" temperature — **normalized to 0–1**
`hum`, `windspeed`	humidity and wind — also normalized
`weathersit`	1 clear · 2 mist · 3 light rain/snow · 4 heavy rain/snow
`workingday`	1 if a weekday that is not a holiday
`casual`, `registered`	rentals by non-members vs. members
`cnt`	total rentals = `casual + registered`

Notice `temp` is **not** in degrees. We will come back to that.

1. Raw data are messy

Before we model anything, we look for what is wrong with the data.

This dataset is often described as "clean." It is not. It has two defects that are worth more than any synthetic example, because they are real.

Defect 1 — the rows that aren't there

Defect 1 — the rows that aren't there

```
hours between first and last observation : 17,544
rows actually in the file                : 17,379
missing                                  : 165

...but how many nulls does pandas report?
0
```

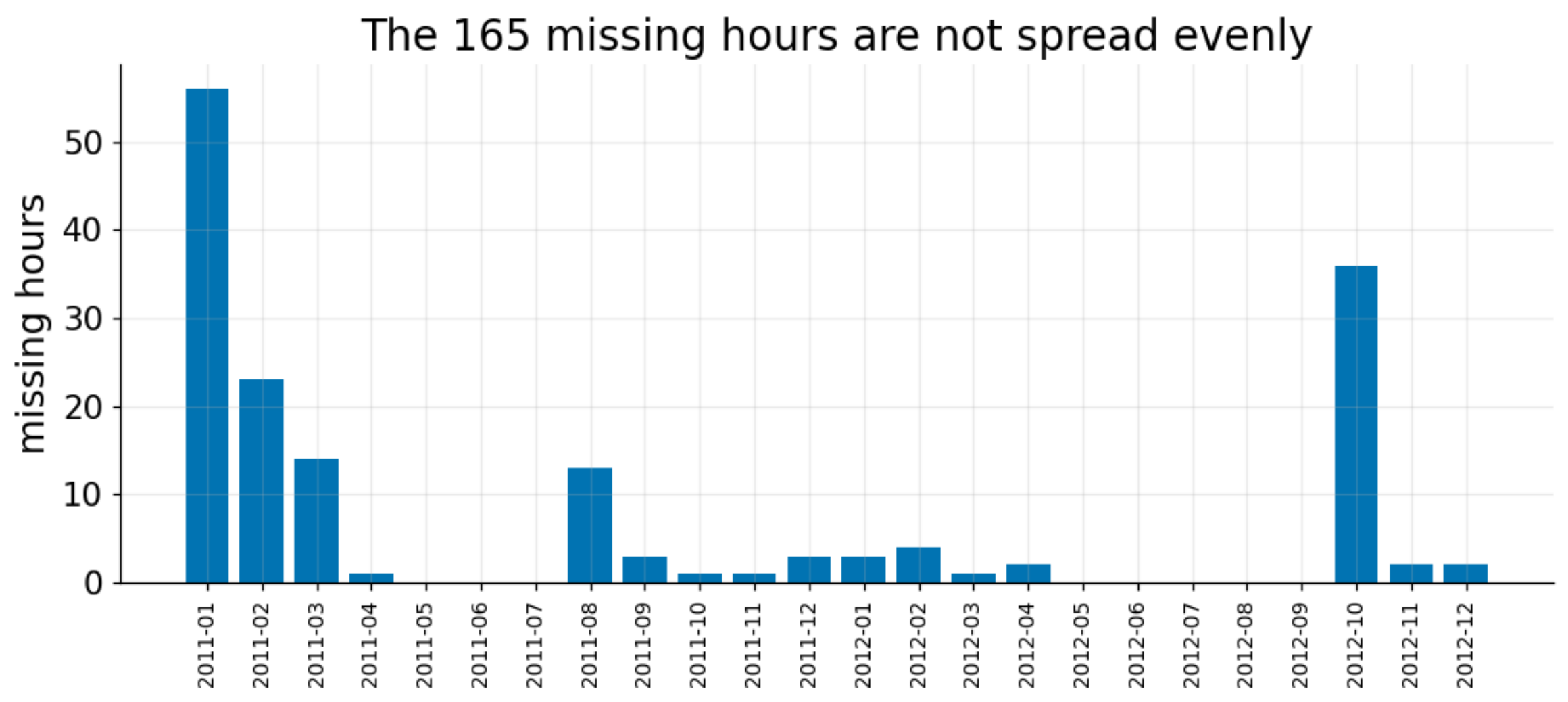
Defect 1 — the rows that aren't there

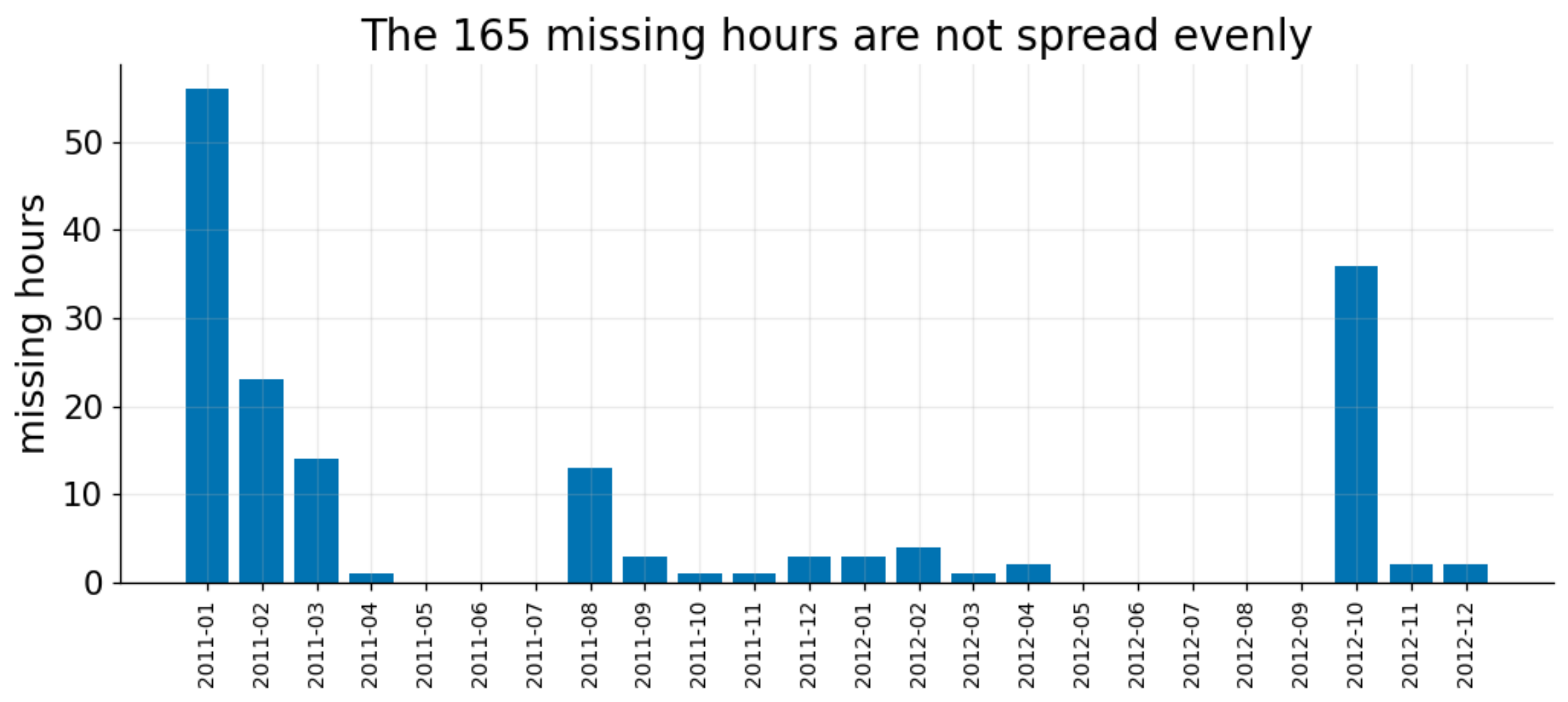
```
hours between first and last observation : 17,544
rows actually in the file                : 17,379
missing                                  : 165

...but how many nulls does pandas report?
0
```

165 hours are missing, and `.isna()` finds zero of them.

The rows are not blank: they are *absent*. A missing value you can see is a nuisance; a missing value you cannot see is a bug in your conclusions. To find these you have to know what a complete dataset should have looked like.

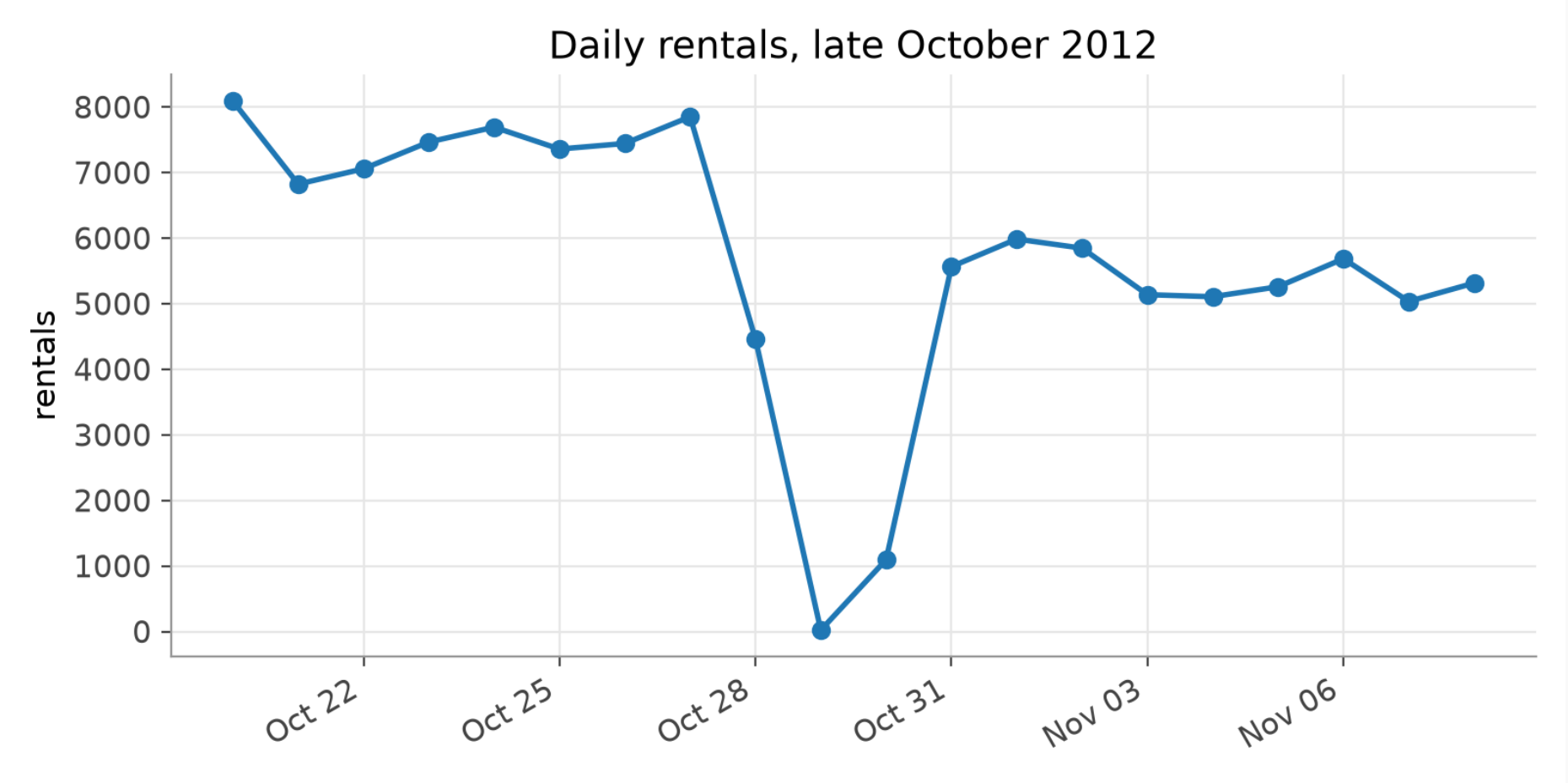

The 165 missing hours are not spread evenly
missing hours
0
10
20
30
40
50
2011-01
2011-02
2011-03
2011-04
2011-05
2011-06
2011-07
2011-08
2011-09
2011-10
2011-11
2011-12
2012-01
2012-02
2012-03
2012-04
2012-05
2012-06
2012-07
2012-08
2012-09
2012-10
2012-11
2012-12



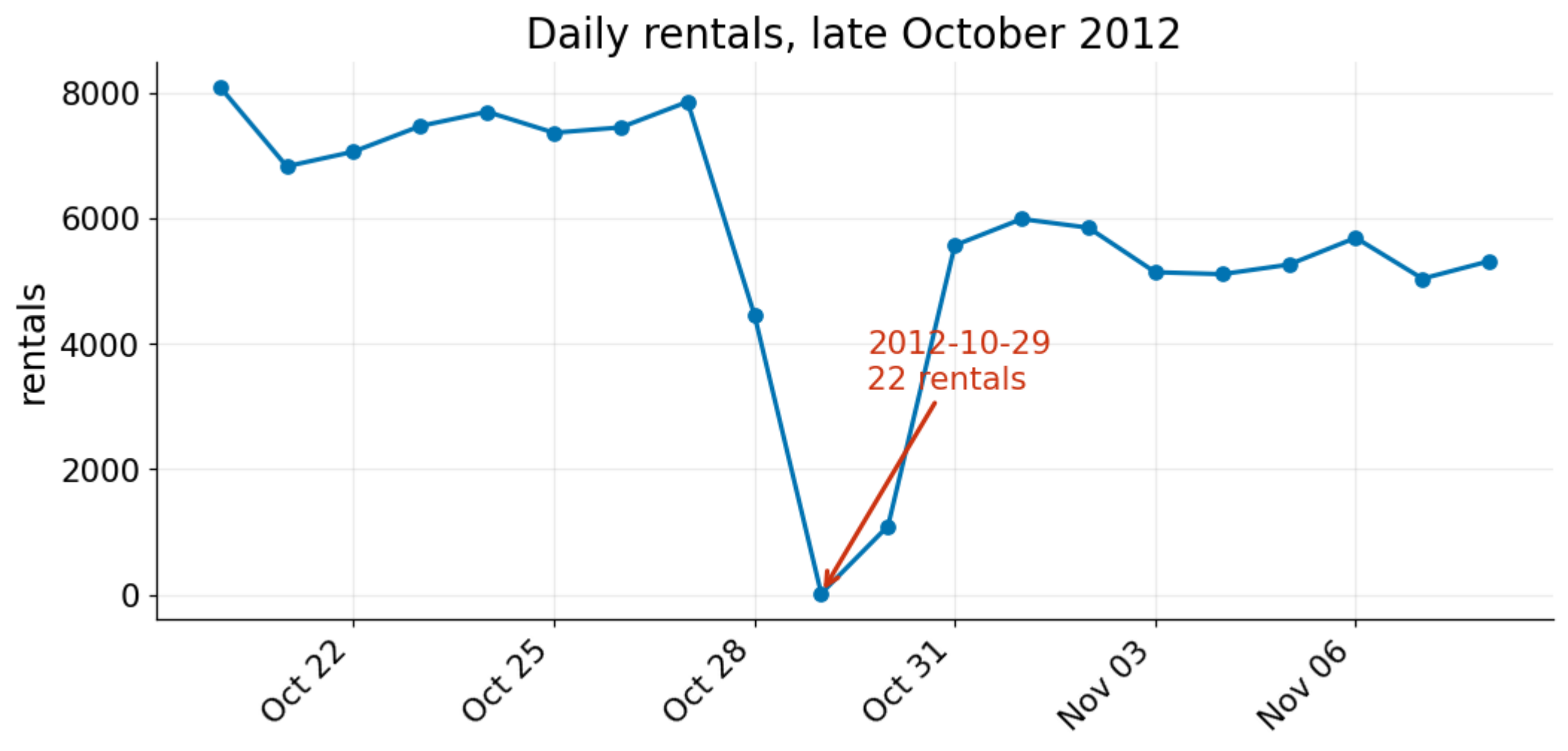
Message: Absent rows are invisible to `.isna()`. Check for what *should* be there.

Defect 2 — the day the system stopped

Defect 2 — the day the system stopped



Defect 2 — the day the system stopped



2. The cleaning pipeline

A pipeline is a repeatable function from raw data to analysis-ready data. Not glamorous. Absolutely central. If this isn't a function, your results aren't reproducible.

The pipeline, in one function

```
def clean(df):
    out = df.copy()
    out["temp_C"]  = -8 + 47 * out["temp"]      # 0.24  ->  3.3 °C
    out["weather"] = out["weathersit"].map({1: "clear", 2: "mist",
                                            3: "light rain", 4: "heavy rain"})
    out["when"]    = out["dteday"] + pd.to_timedelta(out["hr"], unit="h")
    return out

bikes = clean(hour)
```

One function. Same input, same output, every time. That is what makes it reproducible.

```
                 when  hr  temp_C  humidity weather         day_type  casual  \
0 2011-01-01 00:00:00   0    3.28      81.0   clear  weekend/holiday       3
1 2011-01-01 01:00:00   1    2.34      80.0   clear  weekend/holiday       8
2 2011-01-01 02:00:00   2    2.34      80.0   clear  weekend/holiday       5
3 2011-01-01 03:00:00   3    3.28      75.0   clear  weekend/holiday       3
4 2011-01-01 04:00:00   4    3.28      75.0   clear  weekend/holiday       0

   registered  cnt
0          13   16
1          32   40
2          27   32
3          10   13
4           1    1
```

Compare the temperature column before and after:

Compare the temperature column before and after:

raw temp	temp_C	temp_F
0.24	3.3	38.0
0.22	2.3	36.0
0.22	2.3	36.0
0.24	3.3	38.0
0.24	3.3	38.0
0.24	3.3	38.0

Compare the temperature column before and after:

raw temp	temp_C	temp_F
0.24	3.3	38.0
0.22	2.3	36.0
0.22	2.3	36.0
0.24	3.3	38.0
0.24	3.3	38.0
0.24	3.3	38.0

`0.24` is not a temperature. `3.3 °C` is.

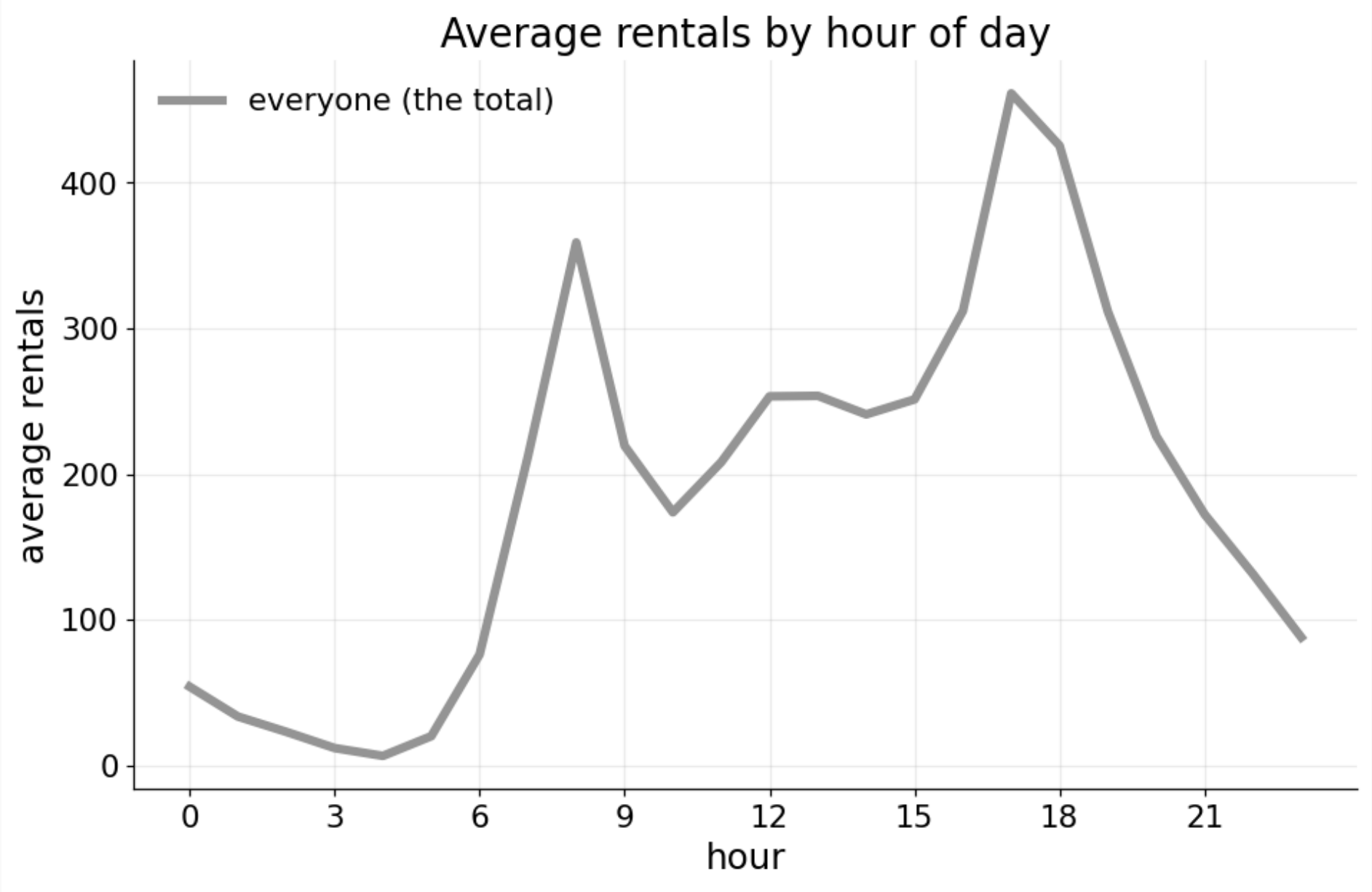
Message: analysis-ready means a human can read a row out loud and it means something. One row = one observation, one column = one variable, in units people use.

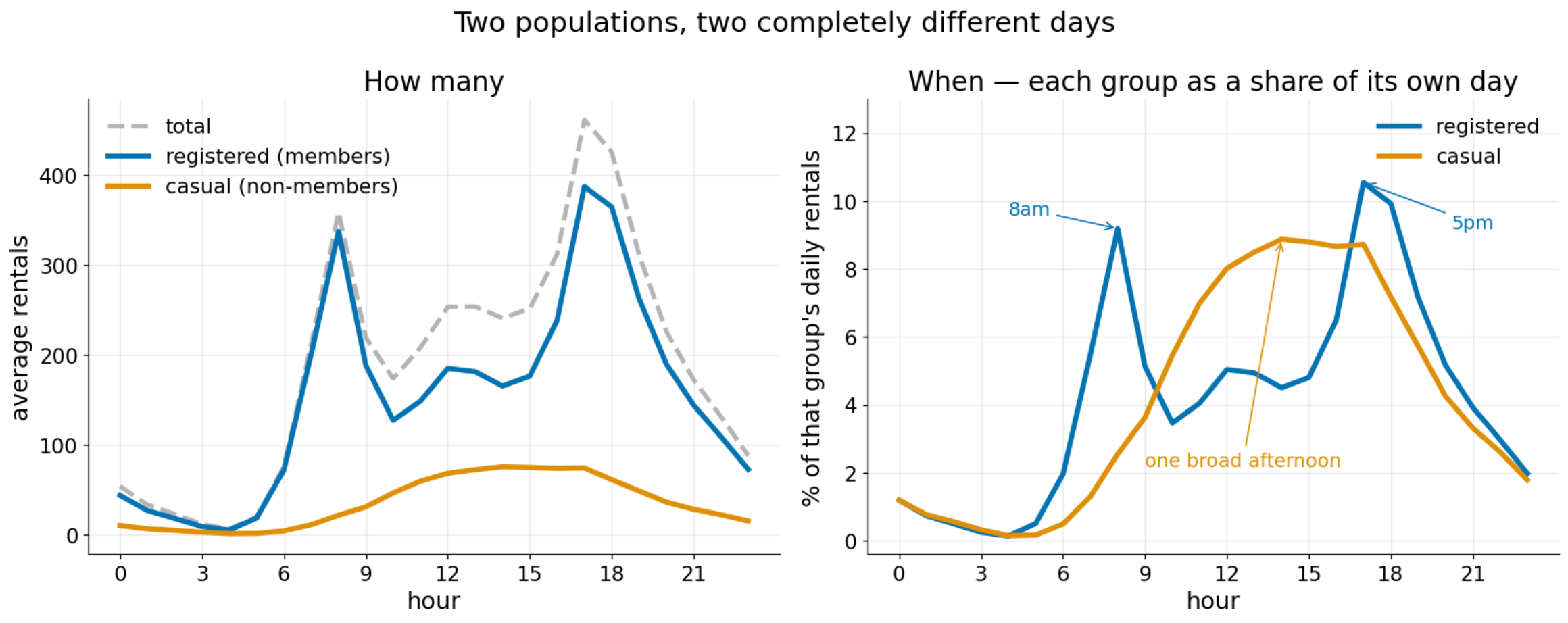
3. Explore before you model

Now we look. Not to confirm anything, to find out what shape the problem has.

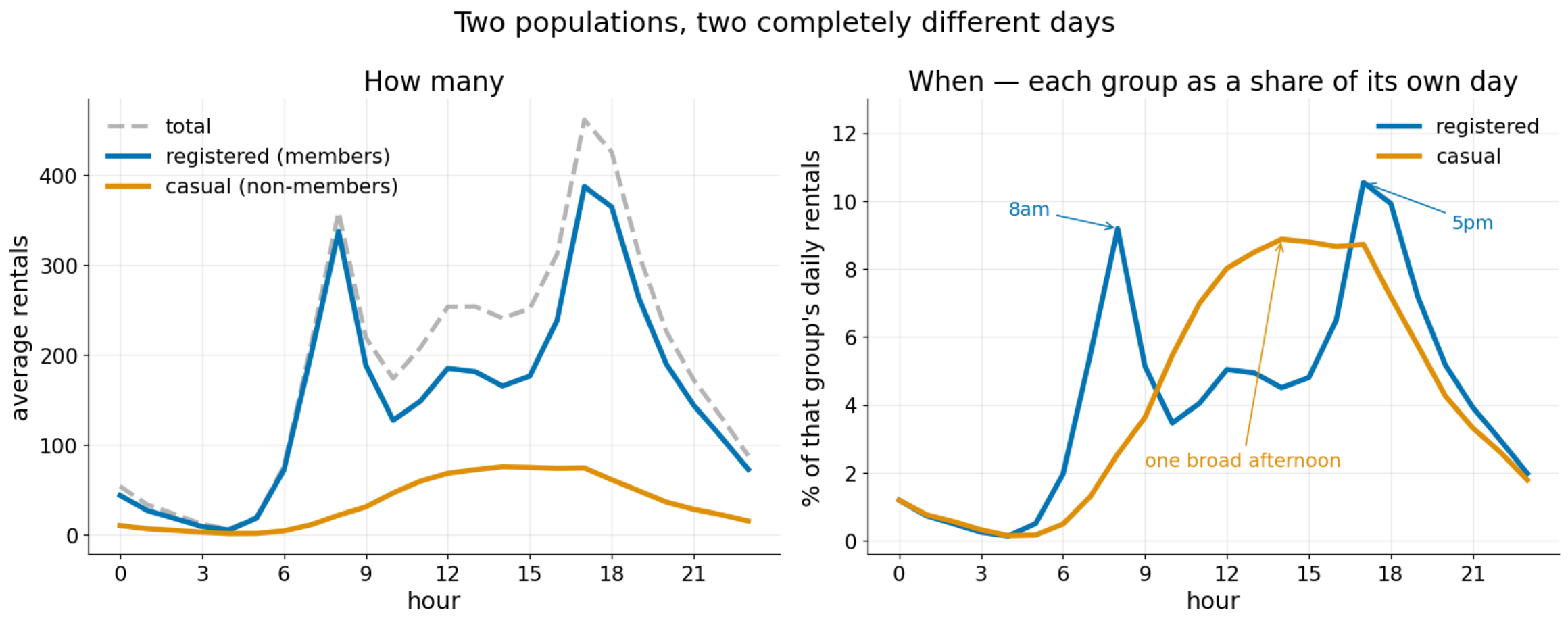
The plot to draw first

Before running this cell: sketch what you think average rentals look like across the 24 hours of a day. Commit to a shape.


Average rentals by hour of day
everyone (the total)
average rentals
0
100
200
300
400
0
3
6
9
12
15
18
21
hour

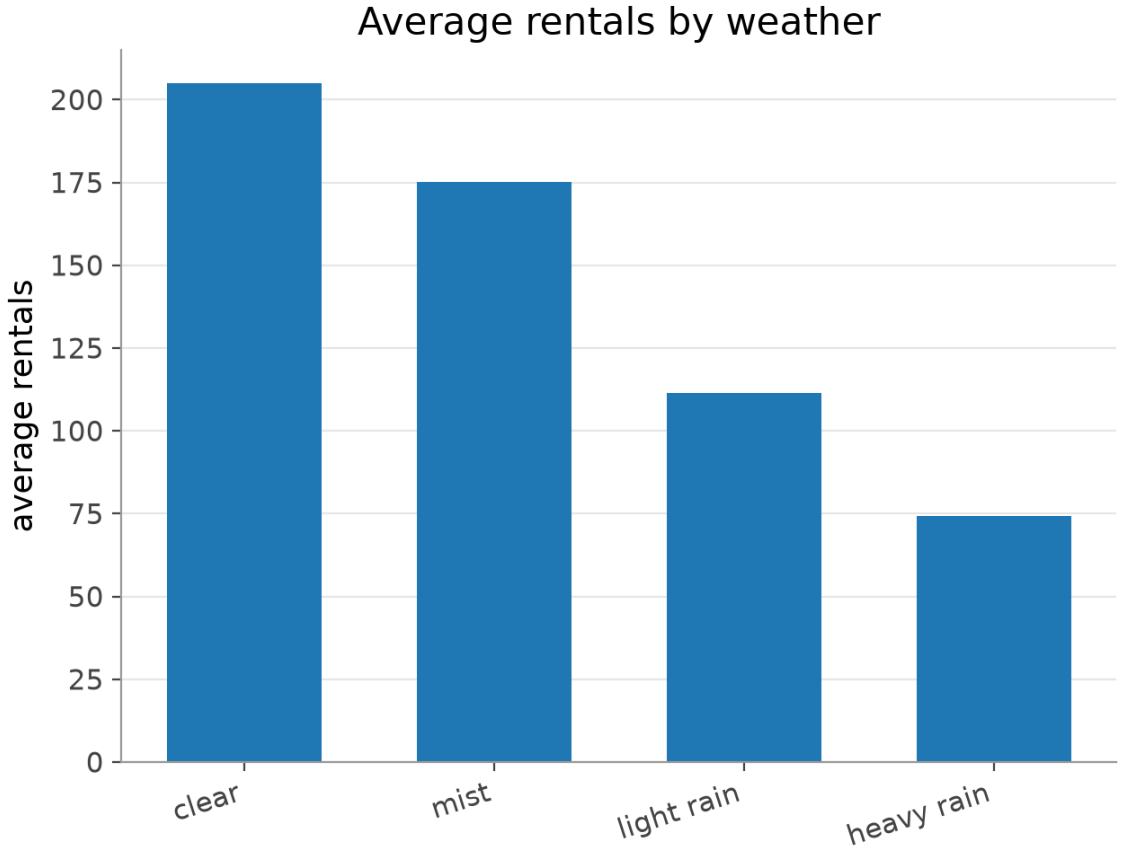


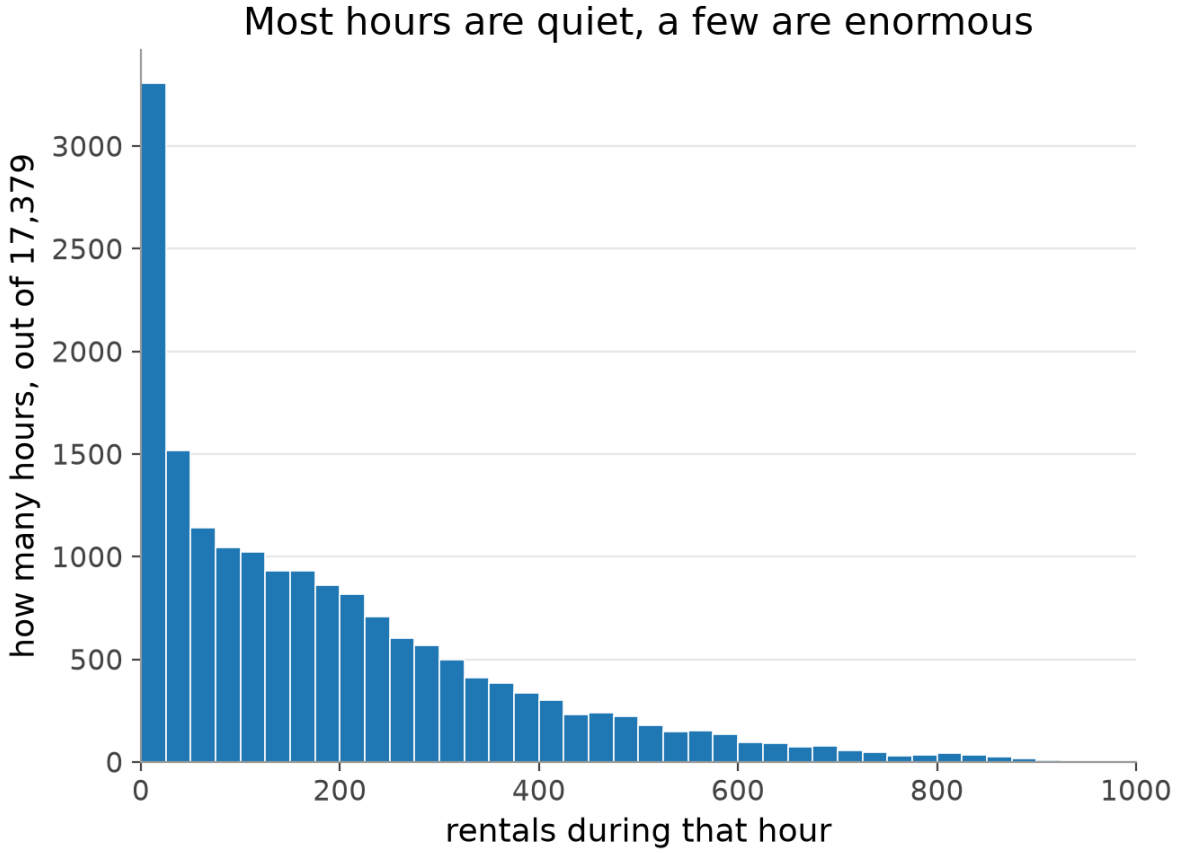
`registered peaks at hour: 17 | casual peaks at hour: 14`

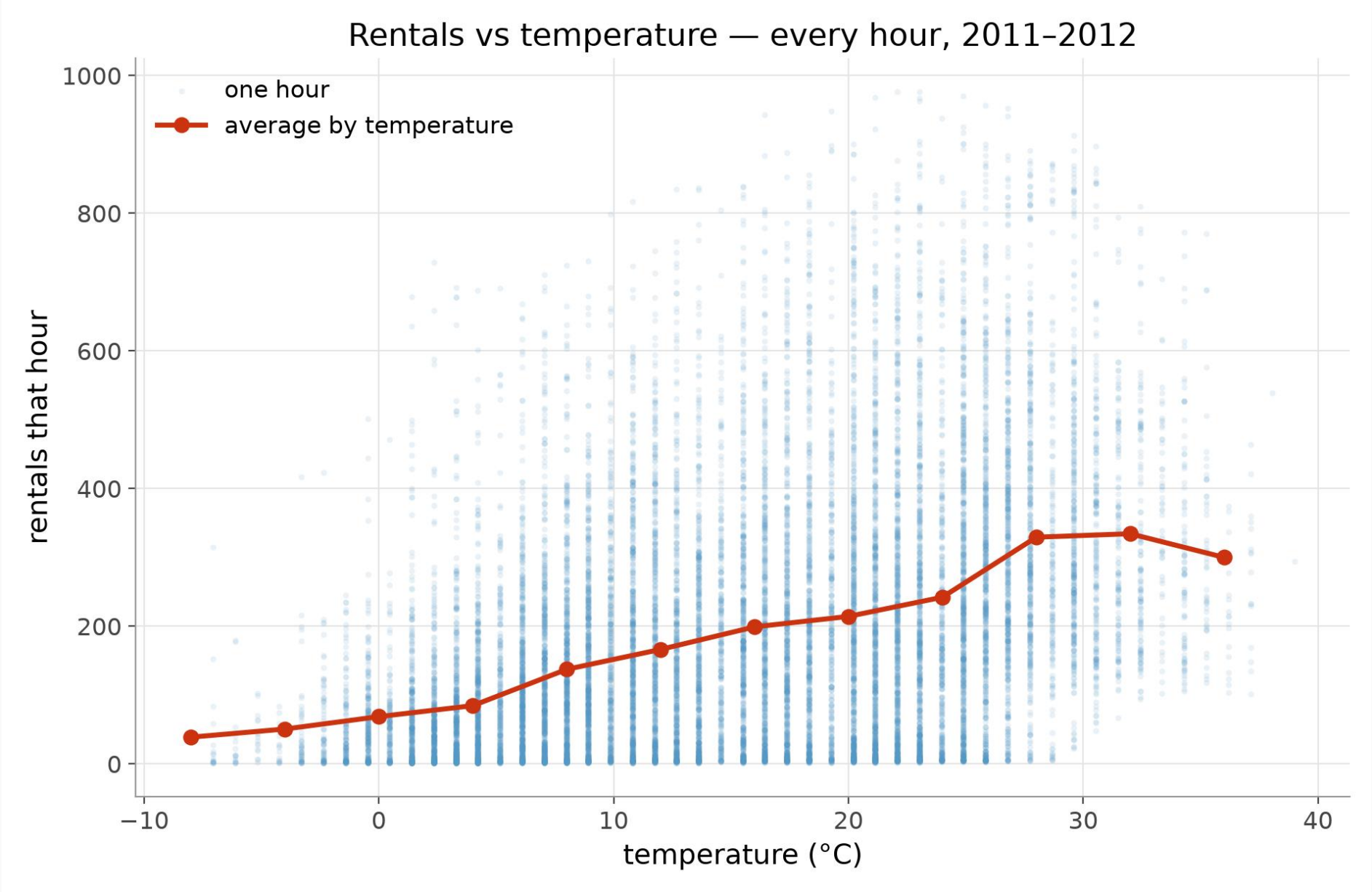



```
registered peaks at hour: 17 | casual peaks at hour: 14
```

Members ride at 8am and 5pm. Casual riders ride at 2pm. **The average is the members' day — they are 81% of every rental — and the casual afternoon has vanished from it.**


Average rentals by weather
average rentals
200
175
150
125
100
75
50
25
0
clear
mist
light rain
heavy rain

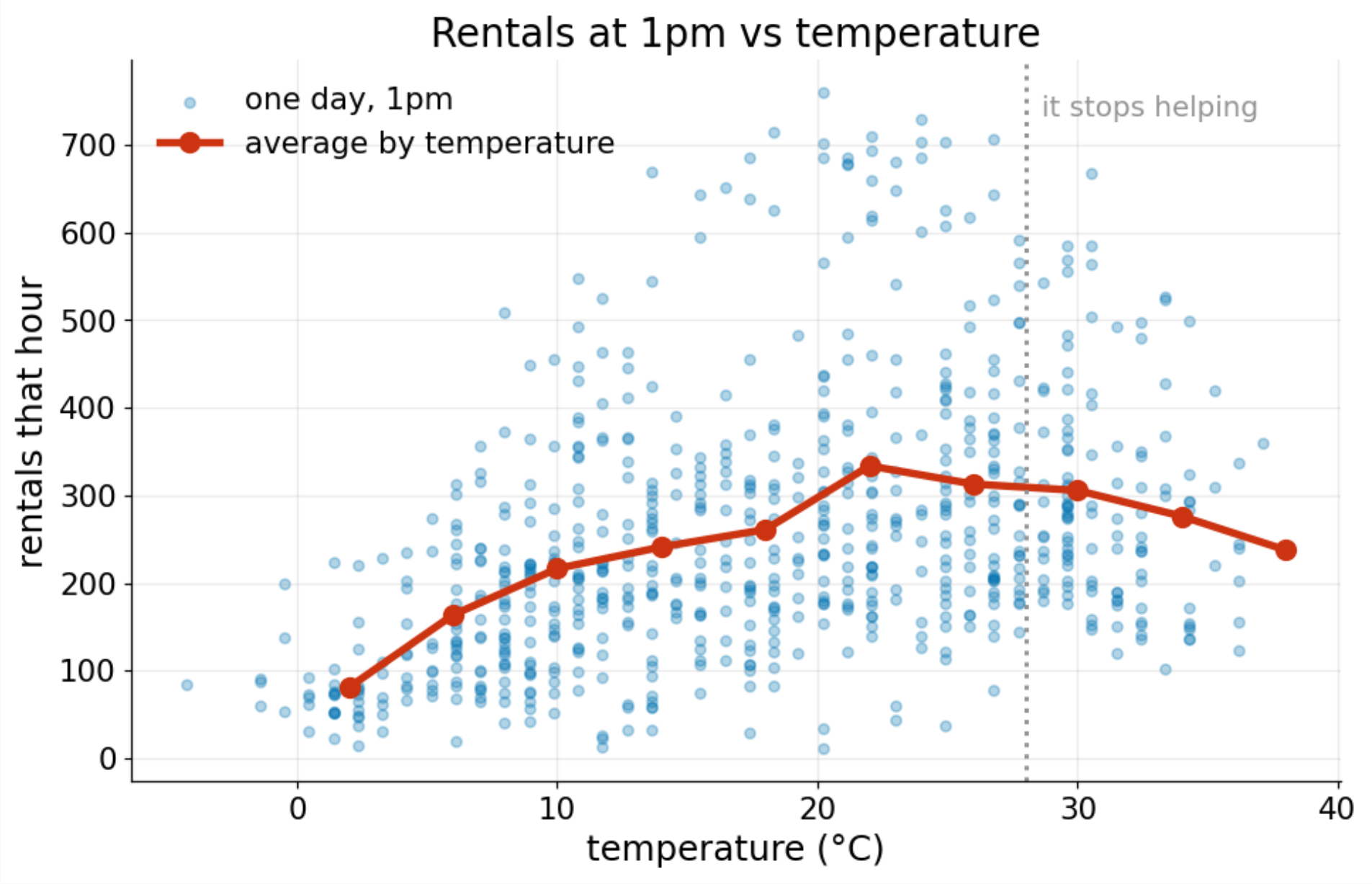

Most hours are quiet, a few are enormous
how many hours, out of 17,379
3000
2500
2000
1500
1000
500
0
0
200
400
600
800
1000
rentals during that hour


Rentals vs temperature — every hour, 2011–2012
one hour
average by temperature
rentals that hour
1000
800
600
400
200
0
−10
0
10
20
30
40
temperature (°C)

4. A hypothesis worth testing

We guessed at the start: *warmer weather means more rentals — but maybe not when it's very hot.*

To test this we need to compare like with like. Comparing 3pm to 3am tells us about the clock, not the thermometer, so we hold the hour fixed and look at **1pm only**.


Rentals at 1pm vs temperature
one day, 1pm
average by temperature
it stops helping
rentals that hour
0
100
200
300
400
500
600
700
0
10
20
30
40
temperature (°C)

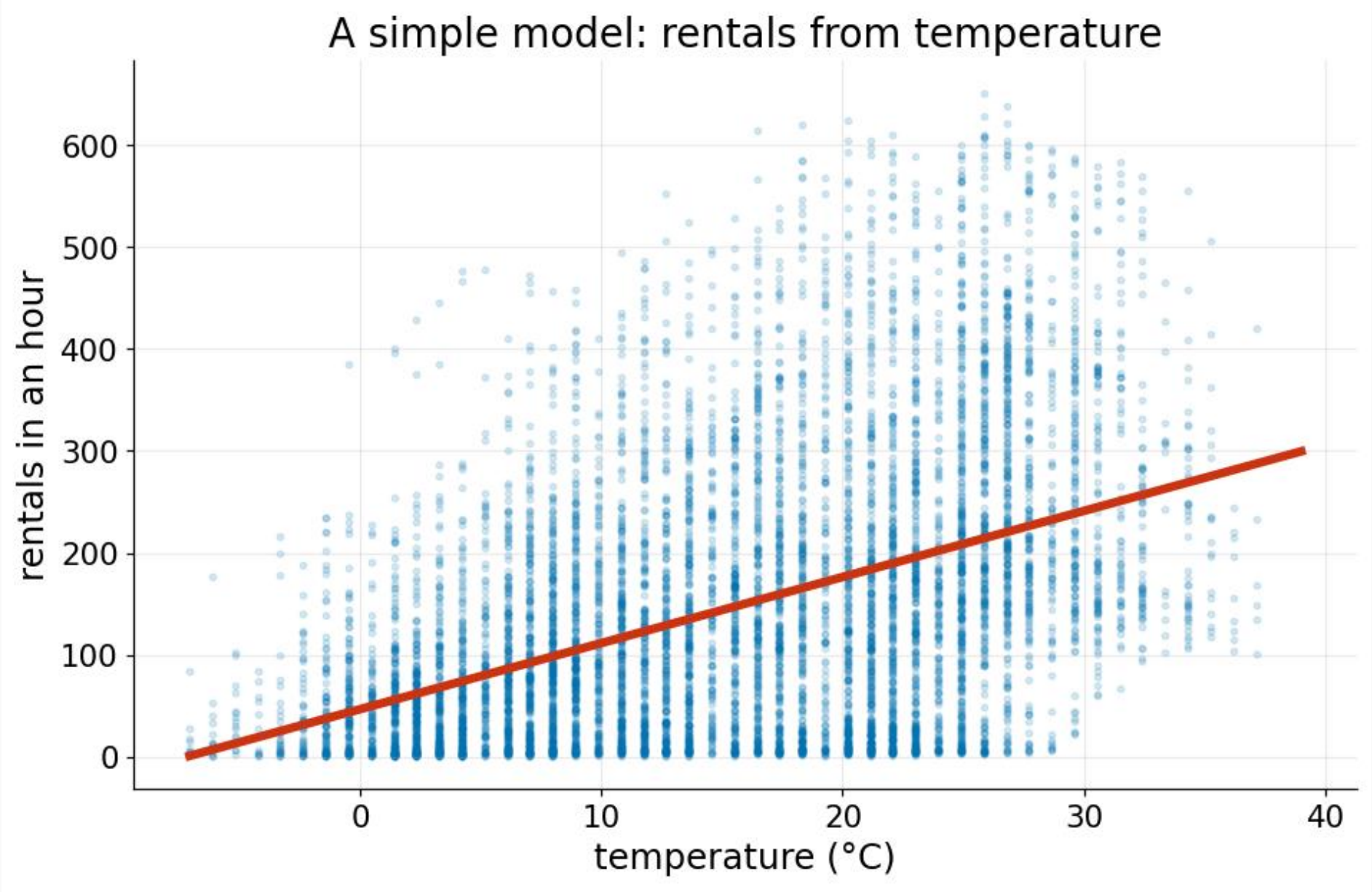
5. A first model, and an honest test

We fit a line: `rentals ≈ intercept + slope × temperature`.

The important question is not "does it fit?" but **"how do we know if it works?"**

We train on **2011** and test on **2012** — because predicting the future is the actual job.

```
slope     : 6.5 extra rentals per °C
train R2  : 0.204
TEST  R2  : -0.026   <- 2012
```


A simple model: rentals from temperature
rentals in an hour
600
500
400
300
200
100
0
0
10
20
30
40
temperature (°C)

A negative R^2 is a gift

R^2 below zero means the model does **worse than always guessing the average**. On 2012 data, this model is worse than useless.

Watch what happens if we had split the data randomly instead:

A negative R^2 is a gift

R^2 below zero means the model does **worse than always guessing the average.** On 2012 data, this model is worse than useless.

Watch what happens if we had split the data randomly instead:

```
random split  test R2 : 0.157  <- looks fine
2011 -> 2012  test R2 : -0.026  <- the truth

2011 average rentals/hour : 144
2012 average, predicted   : 149
2012 average, actual      : 235
```

A negative R^2 is a gift

R^2 below zero means the model does **worse than always guessing the average.** On 2012 data, this model is worse than useless.

Watch what happens if we had split the data randomly instead:

```
random split  test R2 : 0.157  <- looks fine
2011 -> 2012  test R2 : -0.026  <- the truth

2011 average rentals/hour : 144
2012 average, predicted   : 149
2012 average, actual      : 235
```

A random split says $R^2 = 0.157$ and you go home happy. It lies because 2pm and 3pm on the same day are nearly the same row — the test set has already seen the answer.

The honest split shows the real failure: **the system grew.** 144 rentals/hour in 2011, 235 in 2012. The model predicts 149 and is wrong about the world, not about the weather.

6. Two ways a model can be wrong

The line is bad. But *why* it is bad determines the fix, and there are two different diseases with two different cures.

TOO SIMPLE — UNDERFITTING

The model cannot represent the pattern that is there.

Symptom: the errors have a shape.

Cure: give it the variable it is missing.

→ *this section, and §9*

TOO FLEXIBLE — OVERFITTING

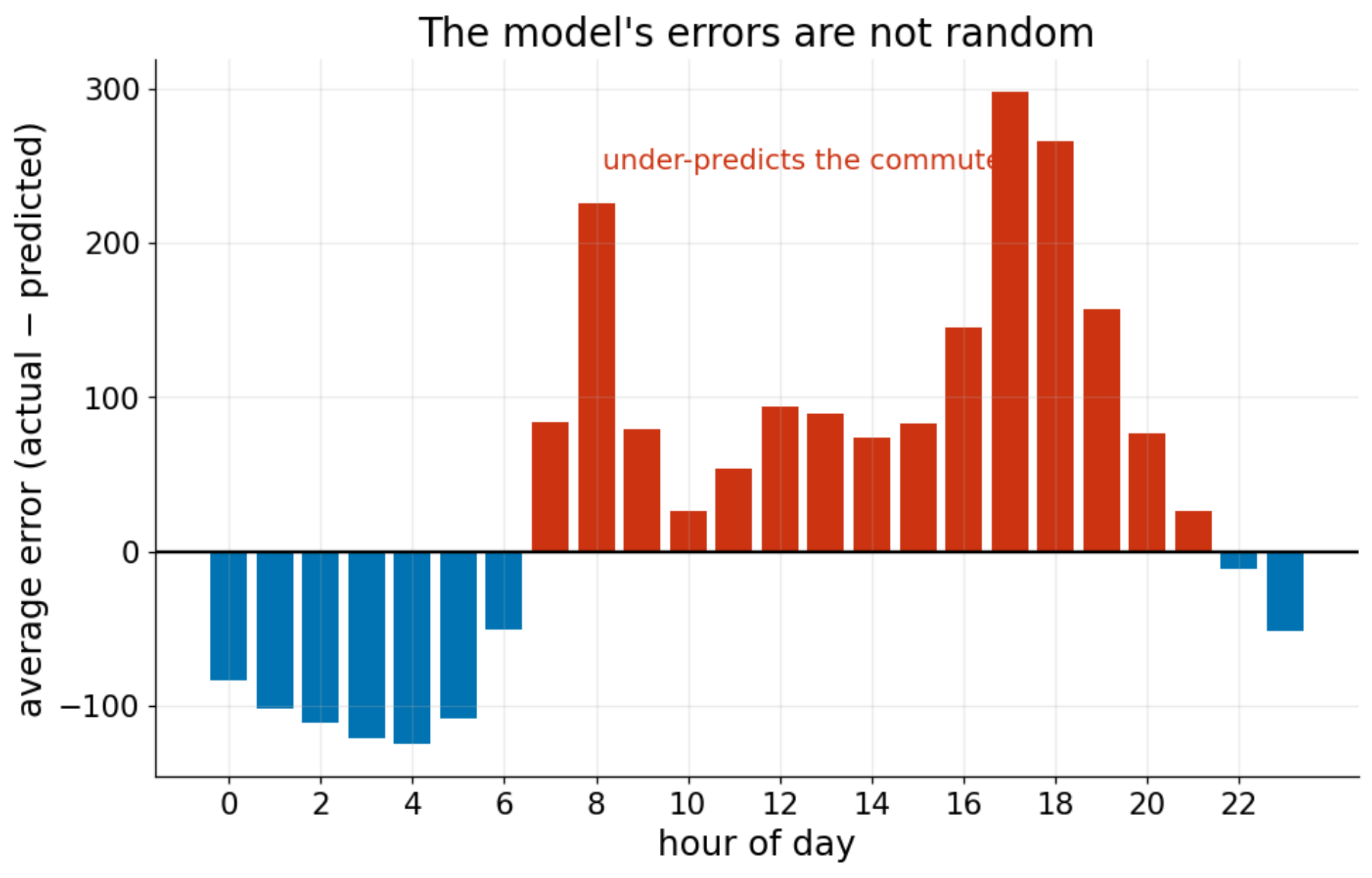
The model memorises the sample it was trained on.

Symptom: great on training data, terrible on unseen days.

Cure: less flexibility, or more data.

→ *§7 and §8*

We take them in order. First one first — look at the model's mistakes, organised by hour of day.


The model's errors are not random
under-predicts the commute
average error (actual − predicted)
300
200
100
0
−100
0
2
4
6
8
10
12
14
16
18
20
22
hour of day

A straight line is the wrong shape

The errors had a shape, so the model is too simple. There are two different ways to fix that:

1 **Give it more variables.**

The clock, the calendar, the weather. → *§9*

2 **Give it a more flexible shape.**

Let the curve bend instead of forcing a straight line. → *next section*

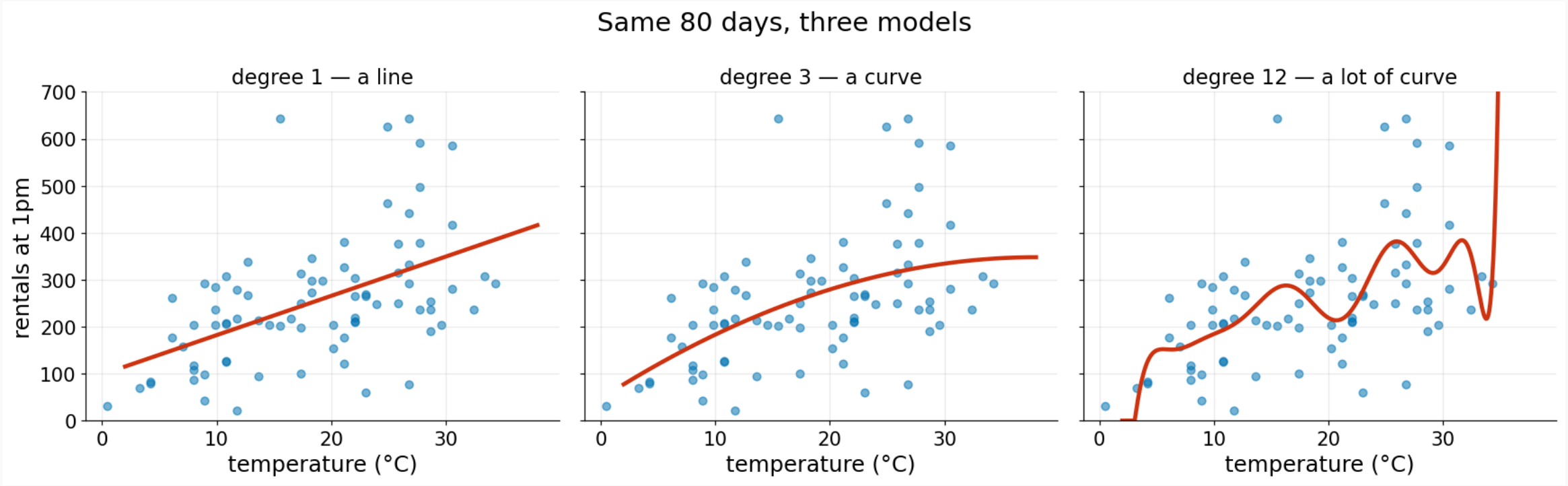
We already know we need it. Back in §4, at 1pm, rentals climbed with temperature and then came back down at the top end. **A straight line cannot do that** — it has one slope and is stuck with it, all the way up.

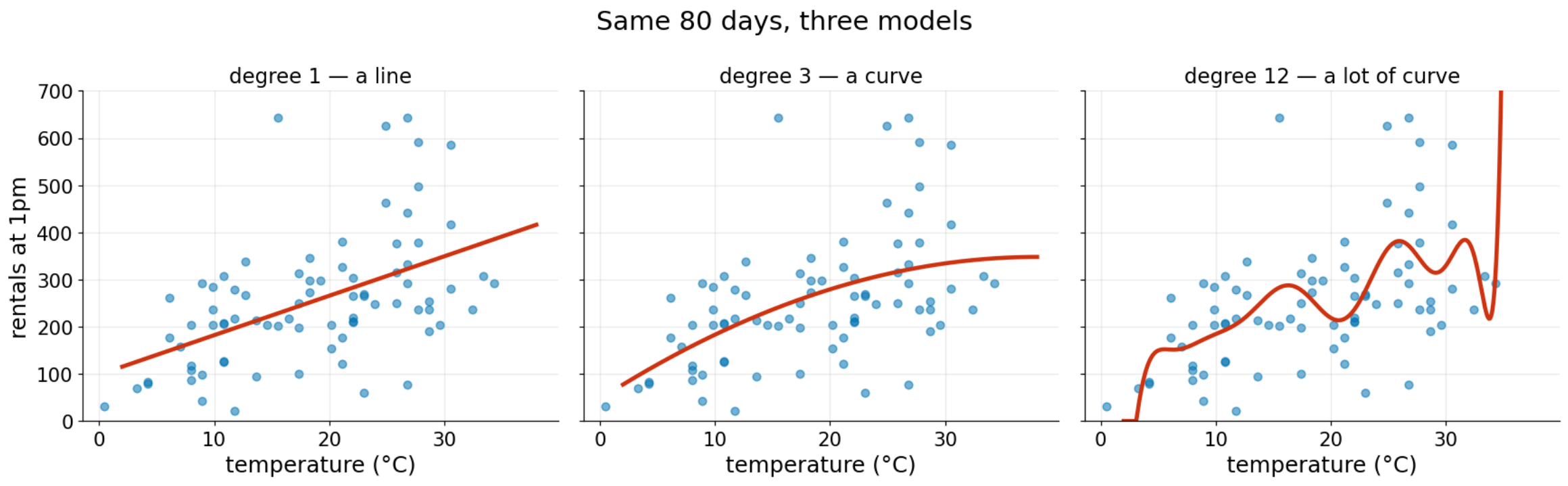
So we let the model bend — and then we find out what happens when we let it bend too much.

7. Flexibility, and too much of it

Now the other disease. We just said a curve is the right answer here — so let's fit one.

We use **80 days at 1pm** — a small sample, deliberately. Overfitting is not about the model alone; it is about the model *relative to how much data you have*.

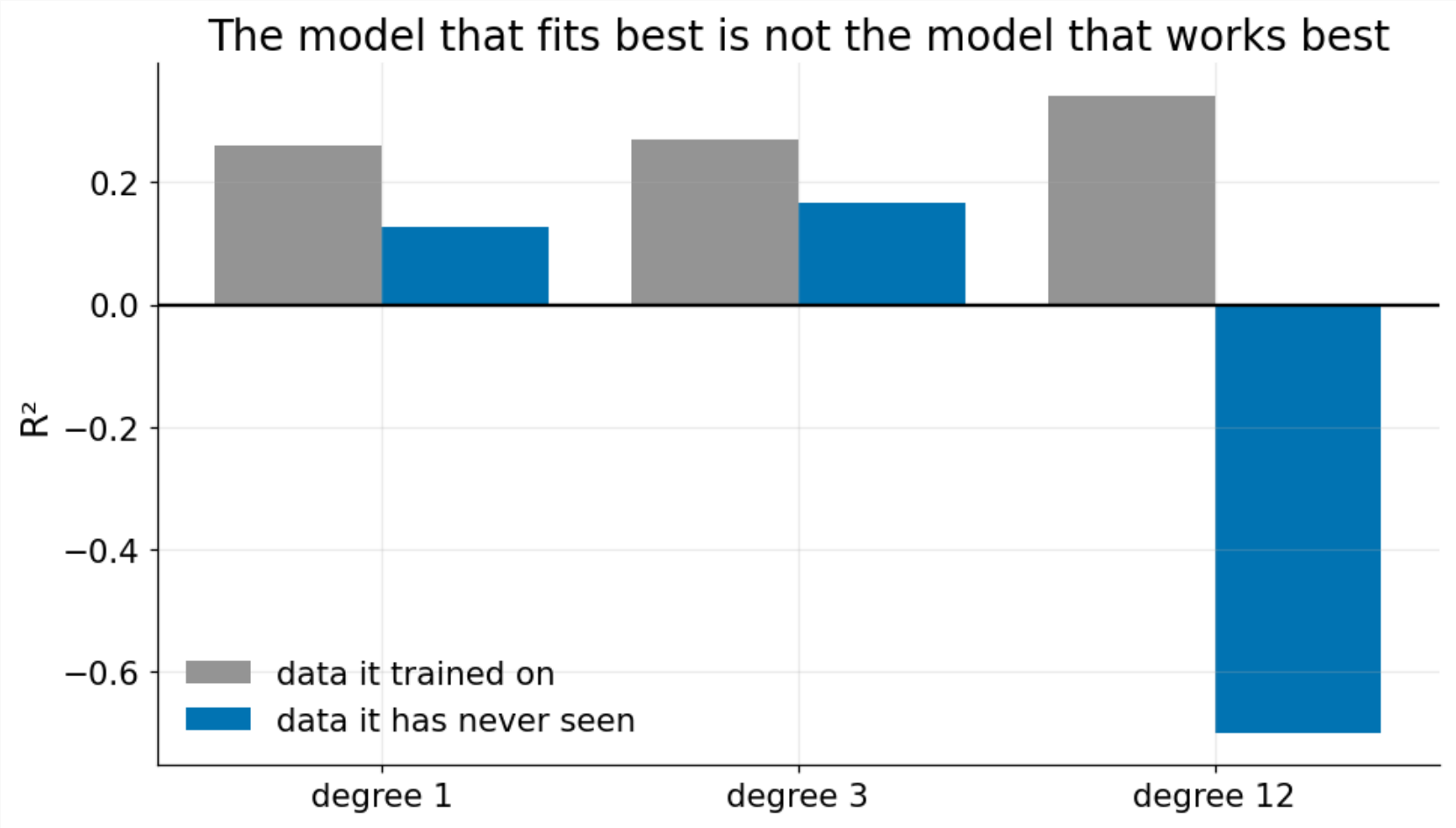

Same 80 days, three models
degree 1 — a line
degree 3 — a curve
degree 12 — a lot of curve
rentals at 1pm
0
100
200
300
400
500
600
700
0
10
20
30
temperature (°C)



Before running the next cell — vote. Which of these three will do best on days it has never seen?

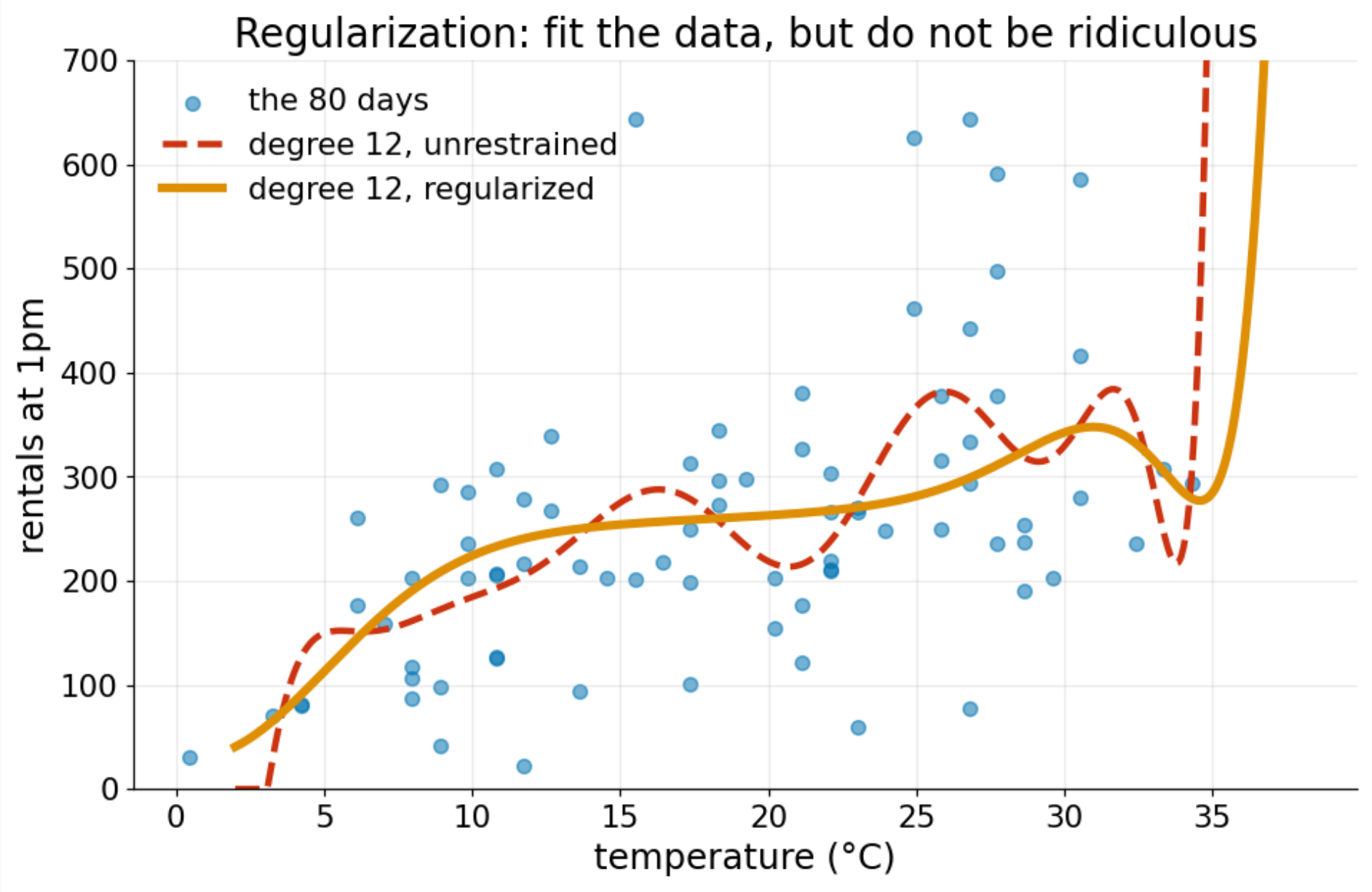
model	R2 on the 80 it trained on	R2 on ~650 unseen days
degree 1 — a line	0.260	0.127
degree 3 — a curve	0.270	0.166
degree 12 — a lot of curve	0.342	-0.700

model	R2 on the 80 it trained on	R2 on ~650 unseen days
degree 1 – a line	0.260	0.127
degree 3 – a curve	0.270	0.166
degree 12 – a lot of curve	0.342	-0.700



8. Regularization: fit the data, but don't be ridiculous

We keep the flexible model (degree 12) and simply add a rule: *large coefficients are discouraged*.
Same model, same data, one extra instruction.


Regularization: fit the data, but do not be ridiculous
the 80 days
degree 12, unrestrained
degree 12, regularized
rentals at 1pm
temperature (°C)
0
100
200
300
400
500
600
700
0
5
10
15
20
25
30
35

9. Add what was missing

Section 6 told us the model was missing the clock. Let's give it the clock, the calendar, and the weather — and score everything honestly on 2012.

model	train R2	TEST R2 (2012)	TEST error (MAE)
temperature only	0.204	-0.026	153.0
+ everything, hour as a number	0.396	0.165	130.0
+ hour as 24 categories	0.677	0.429	109.0
+ regularized (alpha=1)	0.677	0.428	109.0

model	train R2	TEST R2 (2012)	TEST error (MAE)
temperature only	0.204	-0.026	153.0
+ everything, hour as a number	0.396	0.165	130.0
+ev., hour as 24 catego	0.677	0.429	109.0
+ev., regularized (alpha=1)	0.677	0.428	109.0

Look at the jump between rows 2 and 3. **Same variables, same model, same data.**

The only change is how `hr` was written down. As a number, it tells the model that hour 23 is "more" than hour 1 and that demand moves in a straight line across the day. It doesn't — it has two humps. Letting the model treat each hour as its own category takes test R^2 from 0.17 to 0.43 and knocks 21 rentals off the average error.

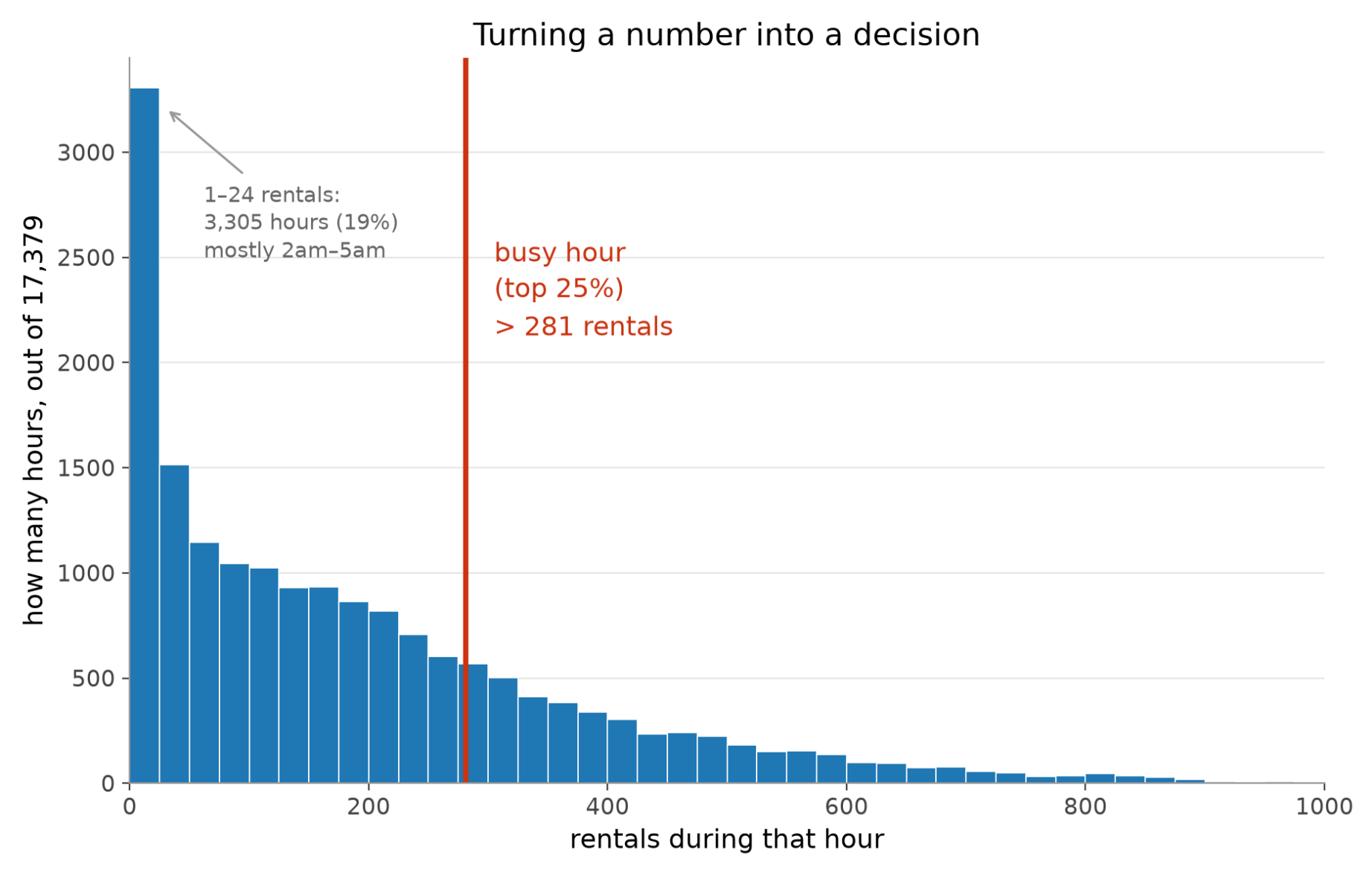
The last row adds regularization and changes nothing, which is the lesson from Section 7 running in reverse: 13,000 rows against 33 features is a comfortable ratio, so there is no excess flexibility left to punish.

Message: how you encode a variable can matter more than which model you pick.

10. Sometimes you only need a category

The operations team doesn't need "347 rentals." They need to know **is this a busy hour or not** — because that's what a dispatch decision looks like.

We define busy as the top 25% of hours. Note that 25% is *our choice*, not a fact in the data.


Turning a number into a decision
1–24 rentals:
3,305 hours (19%)
mostly 2am–5am
busy hour
(top 25%)
> 281 rentals
how many hours, out of 17,379
0
500
1000
1500
2000
2500
3000
0
200
400
600
800
1000
rentals during that hour

The question changed shape

The moment we asked *busy or quiet* instead of *how many*, we stopped doing one kind of problem and started doing another.

UNTIL NOW — REGRESSION

Target a number — 347 rentals this hour

Score R^2, MAE — how far off were we

Models a line, a curve, ridge

§5 - §9

FROM HERE — CLASSIFICATION

Target a label — busy, or quiet

Score accuracy — what share did we get right

Models logistic regression, a tree

§10 - §12

Same rows, same columns, same features. **One different question — and the target, the score, and the models all change with it.**

Logistic regression: a line, on the odds

We want **P(busy)** — and a probability has to stay between 0 and 1. A straight line will not.

$$\log\left(\frac{P(\text{busy})}{1 - P(\text{busy})}\right) = -3.72 + 0.113 \times \text{temp}$$

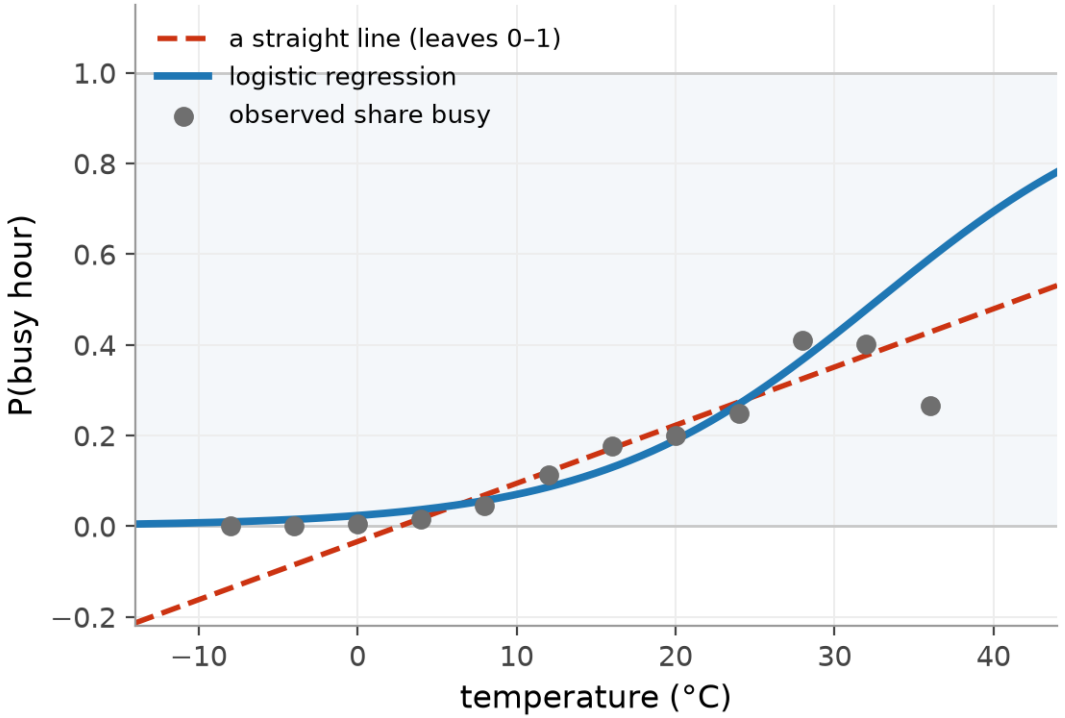
Look at the right-hand side: **intercept + slope × variable** — the same object we fitted in §5. Only what it predicts changed.

The line sits on the *odds*, not the probability — so the reading is multiplicative:

every +1 °C multiplies the odds of a busy hour by **e^0.113 = 1.12**, about 12%. Ten degrees warmer: **three times** the odds.

```
 5 °C  →  P(busy)   4%
15 °C  →  P(busy)  12%
25 °C  →  P(busy)  29%
```

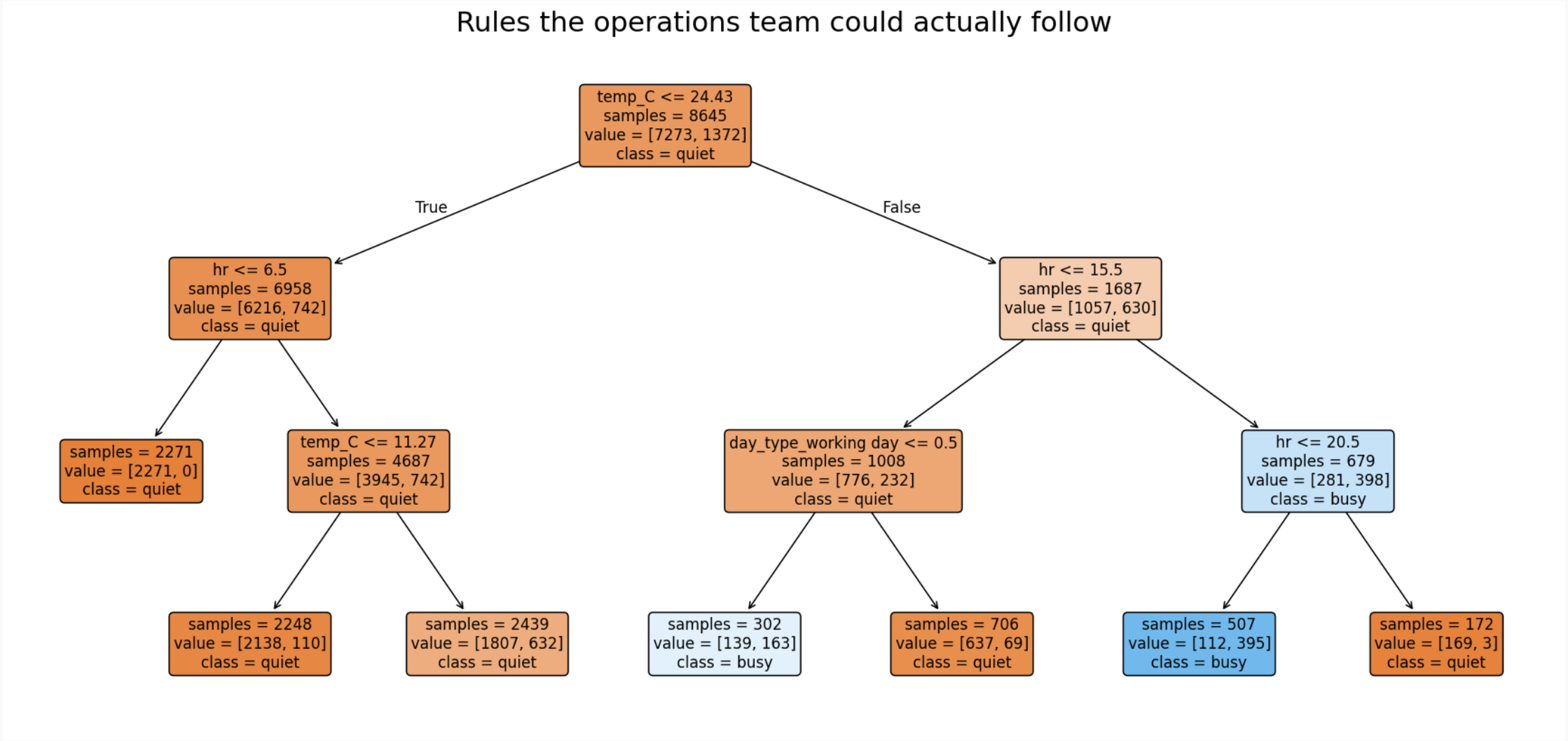
Fitted on 2011, temperature only, all hours.



11. A model that finds the humps by itself

A tree splits the data with simple rules. It doesn't care that hour-of-day isn't linear — it will just cut the day into pieces. And you can read the result out loud.

```
tree accuracy on 2012: 72.9%
logistic regression  : 70.4%
```



The same tree, written out as rules someone could tape to a wall:

The same tree, written out as rules someone could tape to a wall:

```
|--- temp_C <= 24.43
|   |--- hr <= 6.50
|   |   |--- class: quiet
|   |--- hr >  6.50
|   |   |--- temp_C <= 11.27
|   |   |   |--- class: quiet
|   |   |--- temp_C >  11.27
|   |   |   |--- class: quiet
|--- temp_C >  24.43
|   |--- hr <= 15.50
|   |   |--- day_type_working day <= 0.50
|   |   |   |--- class: busy
|   |   |--- day_type_working day >  0.50
|   |   |   |--- class: quiet
|   |--- hr >  15.50
|   |   |--- hr <= 20.50
|   |   |   |--- class: busy
|   |   |--- hr >  20.50
|   |   |   |--- class: quiet
```

12. The 100% problem

Back to the last guess on our list.

Suppose we group riders and ask what fraction come back for a second rental. One neighborhood reports a **100% repeat rate**. Should the city stock it first?

Borrowing strength, in one line

α and β are not knobs to tune. They are **pretend riders** — a tally we act as if we had already seen.

- `α = 3` pretend riders who came back
- `β = 7` pretend riders who did not
- `α + β = 10` how much the belief is worth, in riders
- `α / (α + β) = 0.30` what the belief claims: a 30% repeat rate

```
# prior: the population repeats at 30%, and that belief is worth 10 riders
alpha, beta = 0.30 * 10, 0.70 * 10

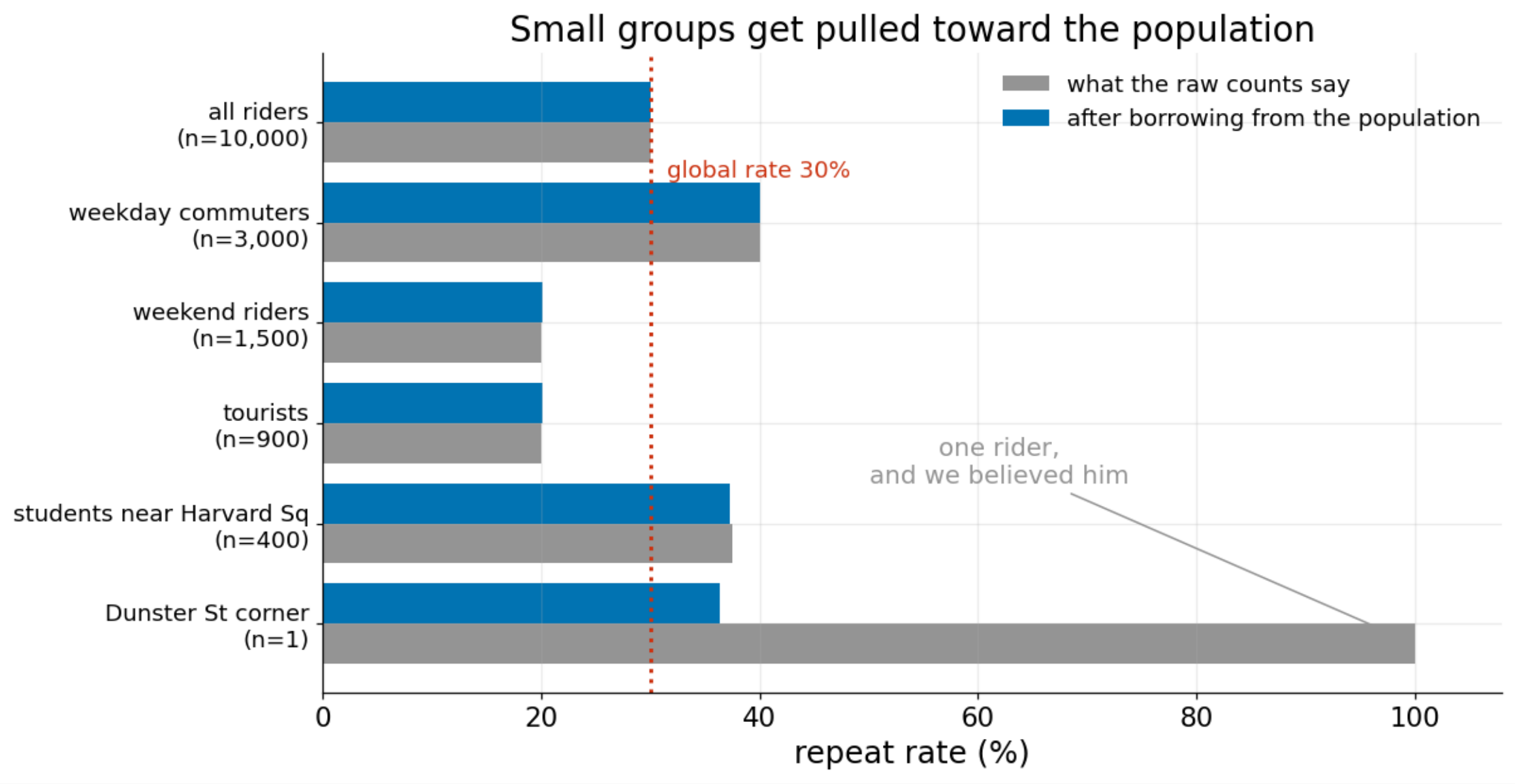
posterior = (repeats + alpha) / (riders + alpha + beta)
```

In other words: **add 3 to the repeats, add 10 to the riders.** Watch it at the two extremes:

```
weekday commuters   (1200 + 3) / (3000 + 10)  =  40.0%   the prior is lost in the noise
Dunster St corner   (   1 + 3) / (   1 + 10)  =  36.4%   3 pretend riders outvote 1 real one
```

`riders` is in the denominator, so the prior fades on its own as the real count grows.

group	riders	repeats	naive %	bayes %	moved by
all riders	10000	3000	30.0	30.0	0.0
weekday commuters	3000	1200	40.0	40.0	0.0
weekend riders	1500	300	20.0	20.1	0.1
tourists	900	180	20.0	20.1	0.1
students near Harvard Sq	400	150	37.5	37.3	-0.2
Dunster St corner	1	1	100.0	36.4	-63.6


Small groups get pulled toward the population
what the raw counts say
after borrowing from the population
global rate 30%
all riders
(n=10,000)
weekday commuters
(n=3,000)
weekend riders
(n=1,500)
tourists
(n=900)
students near Harvard Sq
(n=400)
Dunster St corner
(n=1)
one rider,
and we believed him
0
20
40
60
80
100
repeat rate (%)

The same trap, in our own data

No invented groups this time — the same 17,379 hours, and **busy** as we defined it in §10: the top 25% of hours.

```
# prior: hours are busy 24.9% of the time, and that belief is worth 10 hours
```

group	hours	busy	naive %	bayes %	moved by
all hours	17379	4326	24.9	24.9	0.0
clear	11413	3214	28.2	28.2	0.0
light rain/snow	1419	146	10.3	10.4	+0.1
heavy rain/snow	3	0	0.0	19.1	+19.1
holiday, 3pm, light rain	1	1	100.0	31.7	-68.3

"Heavy rain is never busy" rests on 3 hours in two years. **"This hour is always busy"** rests on one afternoon — 3pm on 8 October 2012.

Message: the two rows that move are the two rows nobody should have trusted. Nothing about that weather is unusual — the only unusual thing is *n*.

13. Scoring our hypotheses

We started with guesses. Science means going back and marking them.

The scoreboard

The same five guesses we wrote down before we opened the file.

hypothesis	we expected	what we found	verdict
Warmer weather	more rentals	§5 — about 6.5 extra rentals per °C	holds
Very hot weather	maybe fewer	§4 — held at 1pm, the curve really does bend back down	holds
Rush hour	more rentals	§3 — true of members (8am, 5pm), not of riders. Casual riders peak at 2pm; the average describes nobody	revised
Weekends	a different shape of day	§3 — one afternoon hump, no commute; §9 — day_type earns its place in the model	holds
Small subgroups	can mislead you	§12 — a 100% repeat rate, from exactly one rider	holds

Four of five survived; one came back more interesting than we guessed. "Rush hour" was never a fact about riders — it was a fact about *members*.

Message: the point of stating hypotheses up front is being able to be wrong about them in public.

14. The part that actually matters

A model that never changes a decision has done nothing.

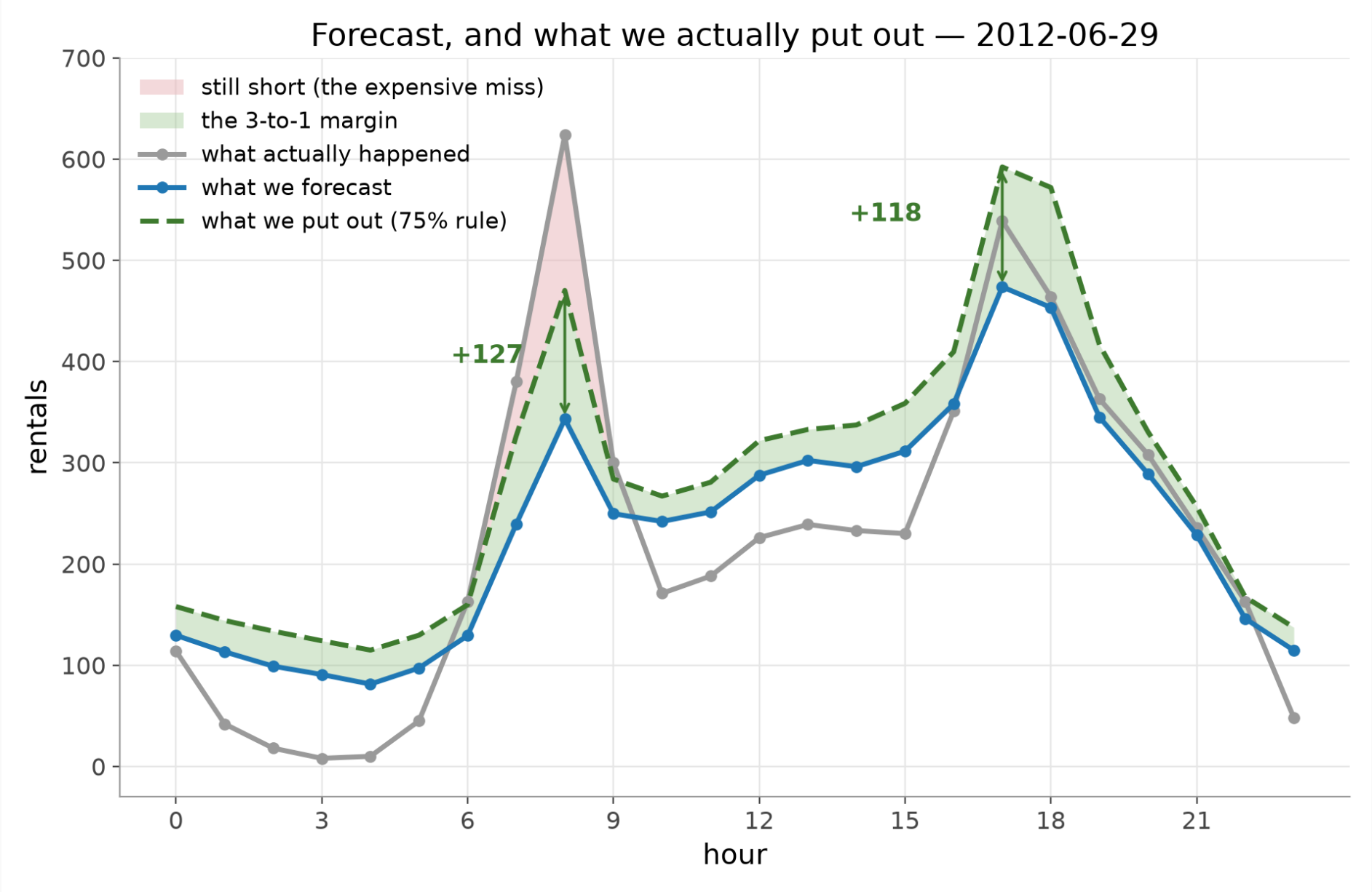
So: it is a spring evening in 2012. Overnight we decide how many bikes to put on the street tomorrow. **How many — and at which hours?**

14. The part that actually matters

A model that never changes a decision has done nothing.

So: it is a spring evening in 2012. Overnight we decide how many bikes to put on the street tomorrow. **How many — and at which hours?**

The costs are not symmetric. A rider who finds an empty rack gives up and the city loses a trip. A rider who finds a full dock has to hunt for another — annoying, but they still ride. Say an empty rack costs **3×** what a full dock costs.


Forecast, and what we actually put out — 2012-06-29
still short (the expensive miss)
the 3-to-1 margin
what actually happened
what we forecast
what we put out (75% rule)
+127
+118
rentals
hour
0
100
200
300
400
500
600
700
0
3
6
9
12
15
18
21

The whole cycle, on one dataset

we did	it turned out
asked a question	how many bikes, and when?
got data	17,379 hours of a real city
found it messy	165 invisible rows, one hurricane
explored	two populations hiding inside one average
modeled	a line — worse than useless on next year
diagnosed	errors with a shape = a variable we forgot
over-corrected	degree 12 memorised 80 days
controlled it	regularization
changed the question	how many → busy or not
distrusted small data	100% was one rider
decided	over-supply, because empty racks cost more

The model is not the goal. The goal is to answer a question well enough to help someone make a better decision.

Everything we skipped past in one line each — least squares, cross-validation, priors, ensembles, causality — is a week of this course.

Thank you!

